Weekly Meeting Update

Sprint Progress & Walking Skeleton Milestone

Over the past week, we achieved approximately 90% architectural completion across our five core services towards an end-to-end walking skeleton. While we postponed the end-to-end live demo today pending two final integration pieces, our multi-module builds, centralized CI pipelines, and individual service components are up and working, with a baseline of 68 passing tests on main.
We also aligned integration and review workflows across repositories. Centralized CI is live and green across repos via reusable GitHub Actions workflows in the `.github` repository. Code reviews resolved smoothly over the weekend, including reconciling POM duplicates, aligning Kubernetes pod tracking labels between the scheduler and tracker, and finalizing MySQL schemas.

PR Reviews & In-Flight Work

- **PR #13: PROD-1 Synthetic CSV Generator & Redis Publisher (`job-orchestrator`): Ready to Merge**
    - Jonah implemented the Section 6.1 synthetic payment CSV generator, flat-memory streaming writer, initial `QUEUED` MySQL persistence, and version 1 `JobMessage` Redis Streams publisher. Rebased on `main` with green tests; awaiting Aaron's re-approval.
- **PR #12: CONSUMER-1 Redis Stream FIFO Dequeue & Concurrency Limiter (`job-orchestrator`): Merged**
    - Jay delivered Lettuce Redis Streams consumer groups, dispatch loops, capacity checks, and MySQL `DISPATCHED` state transitions.
- **PR #14: SCHED-1 Kubernetes Job Spec Builder & ClusterClient (`job-orchestrator`): Merged**
    - Aaron implemented the Fabric8 Kubernetes Job builder with `backoffLimit: 0`, shared `/data` hostPath volume mount, and HTTP 409 idempotency handling on redeliveries.
- **PR #15: TRACKER-1 Kubernetes Pod Event Watcher & Status Persistence (`job-orchestrator`): Merged**
    - Andrew built the Kubernetes pod event watcher, state reconciler (`RUNNING` -> `SUCCEEDED` / `RETRY_WAIT` / `DEAD_LETTERED`), and MySQL audit logging (`pod_event`, `job_attempt`) backed by 45 unit tests.
- **PR #3: INFRA-2 MySQL V1 DDL Migrations (`platform-infra`): Merged**

    - Andrew finalized relational schemas and init scripts (`job`, `job_attempt`, `pod_event`, `account_payment`, `file_progress`) with `updated_at` triggers.
- **PR #16: CORE-2 StorageClient Interface Contract (`job-orchestrator`): In Review**
    - Kenneth opened overnight to define the `StorageClient` interface in `common`. Pending review by Jonah immediately after the meeting.
- **Issue #3: WORK-2 Streamed CSV Parser & Batched MySQL Loading (`batch-workload`): In Progress**
    - Kenneth is building the worker container to stream CSV records line-by-line and batch-insert into `account_payment`. Serves as the primary blocker for the end-to-end walking skeleton.
- **PR #10 & .github: CI-1 Centralized Reusable CI Pipeline: Merged**
    - Jonah established organization-wide reusable GitHub Actions workflows (`reusable-ci.yml`) for multi-module Maven test validation.

Next Tasks on the Board (`MVP Semester 1`)

With the individual service components largely complete, we are queuing up the next phase of pipeline integration and orchestration resilience tasks:

- **CORE-3: Service Application Entrypoints & Local Configuration (`job-orchestrator`)**
    - Wire `ConsumerService` and `Fabric8JobScheduler` into executable application entrypoints once `WORK-2` and `PROD-1` are merged.
- **INFRA-3: End-to-End Walking Skeleton Demo Script (`platform-infra`)**
    - Implement single-command orchestration (`demo-walking-skeleton.sh`) to provision `/data`, start Docker/Kind, trigger the producer, and verify end-to-end MySQL persistence.
- **Retry Policies & Exponential Backoff**
    - Add configurable exponential backoff retry policies across job dispatch and pod execution.
- **Janitor & Requeuer Service**
    - Implement background reconciler service to poll and re-enqueue jobs in `RETRY_WAIT` state back into active streams.
- **Dead-Letter Queue (DLQ) Handling**
    - Route permanently failed or unparseable jobs to a dead-letter queue with audit event logging.
- **Storage Client Integration (S3 / RustFS)**
    - Integrate the `CORE-2 StorageClient` contract with S3/RustFS storage backing for payment files.